

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Introducción a la programación 2



Manual Técnico Proyecto 2

202330209 Milton Efrén Ajxup Ramos

INTRODUCCIÓN

Objetivo

Otorgar al usuario el soporte y ayuda necesaria para que pueda hacer una correcta ejecución de la aplicación de java del estudiante Milton, que en esta ocasión es el archivo del Proyecto 2 para el laboratorio de Introducción a la Programación y Computación 2.

Requerimientos

Tener una versión de java instalada

Tener disponible el archivo .war proporcionado por el estudiante.

Esta es una aplicación web que trabaja con un servidor de Angular para encender este primero desde la terminal se debe estar en la carpeta que contiene el archivo de angular del Proyecto y ejecutar el comando `ng serve`, esto activará el servidor y otro servidor de llamado Tomcat que se ejecuta dentro de un IDE compilando el archivo WAR del proyecto, así que hay que tener ambos instalados, para ejecutar el archivo, se accede en la web a <http://localhost:4200>, esto llevará al proyecto listo para ejecutarse.

Diagrama Entidad Relación

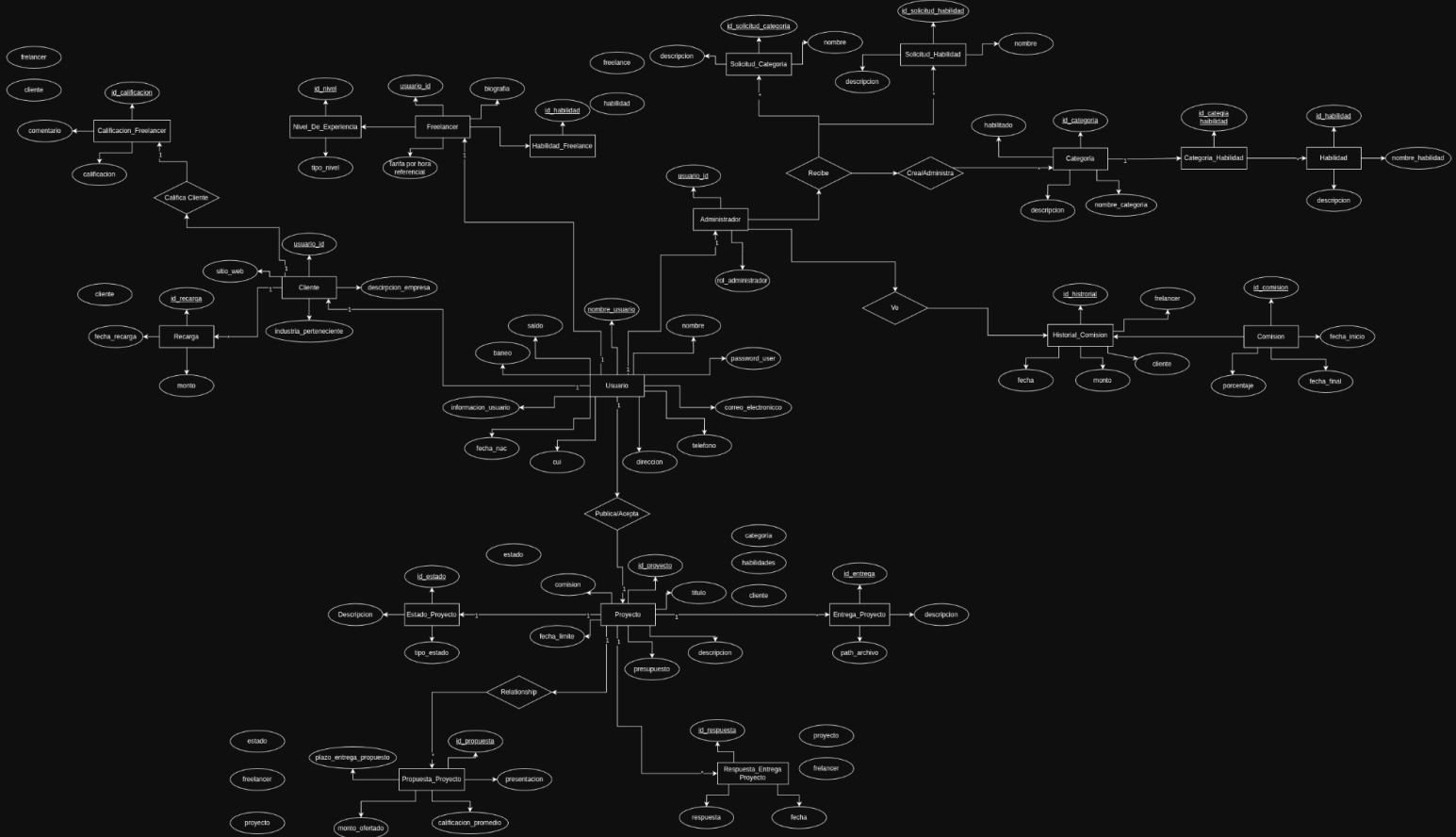
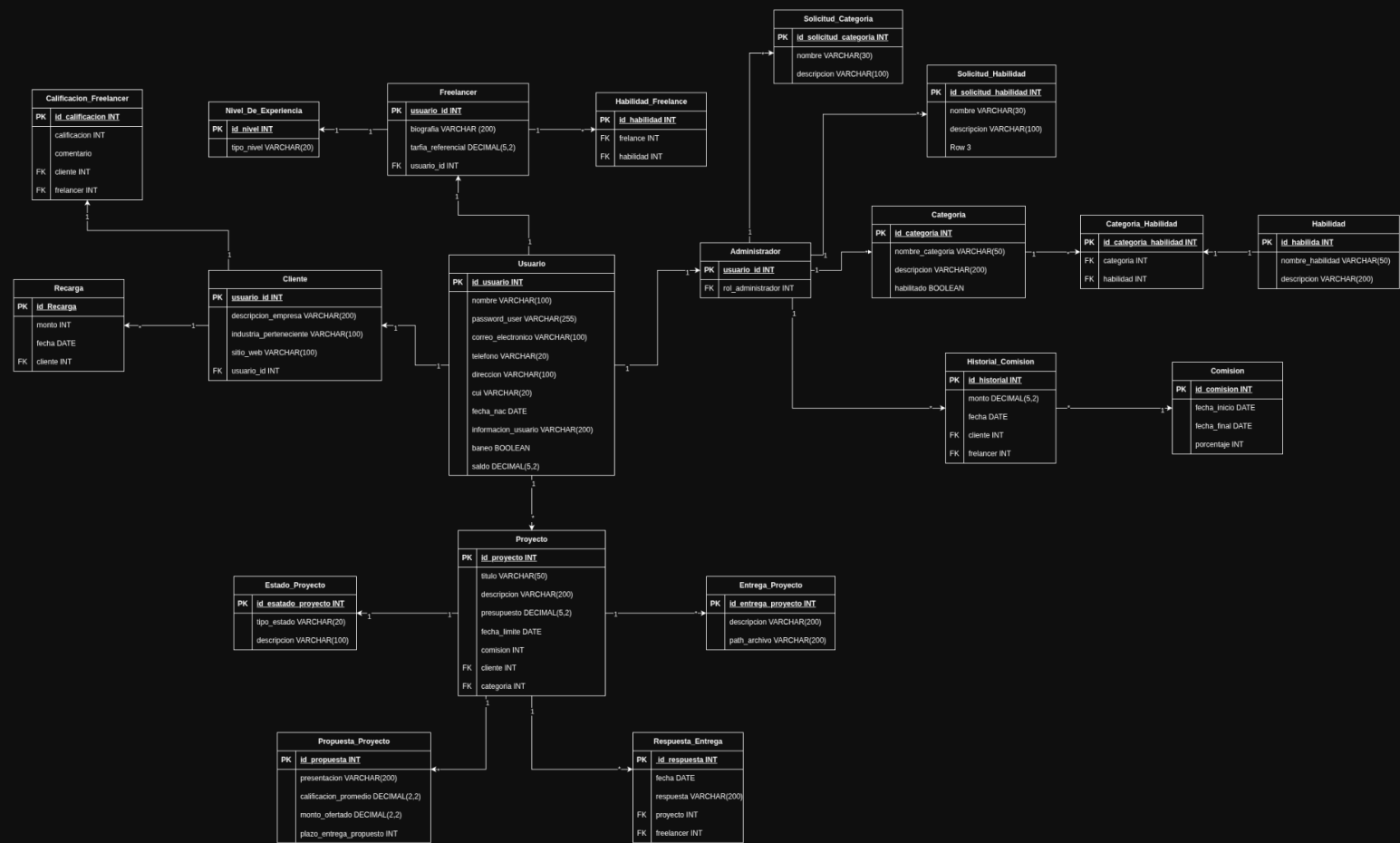


Diagrama de Tablas



Manual Técnico

Descripción General de Clases Importantes

Servlets de Java	Son los encargados de interactuar con la base de datos, la aplicación web manda peticiones para buscar elementos o insertar datos y los servlets reciben esta información y hacen lo pertinente
DAOs del sql	Son las clases encargadas de hacer las consultas con la base de datos, estas clases buscan y arman las listas con los elementos solicitados
Servicios	No en todos los objetos se añadió un servicio, porque no se ameritaba, pero en los objetos como en el login o en usuario, eran los encargados de realizar las acciones correctas con la información que tenía.
Angular	Ordena la información recibida desde el backend

1. Servlets de Java

Estos servlets son peticiones http recibidas desde, en este caso, de angular el cual manda peticiones de tipo get, post, put o delete según como se quieran manipular los objetos de la base de datos.

1.1. Peticiones de tipo Get

La petición de tipo get se hace de forma que tiene que recibir en que se espera que devuelva un valor o un arreglo de valores, esta petición no permite enviar un objeto como body, en su lugar, se opta por mandar instrucciones en la misma petición http, concatenando elementos string separando con "/", y en el backend hay que encargarse de filtrar que es lo que se quiere, por ejemplo:

```
public getReservacionesEntreFechas(fechaInicial: string, fechaFinal: string):  
Observable<Reservacion[]> {
```

```
return this.httpClient.get<Reservacion[]>
(`${this.restApi.getApiURL()}reporte/entre-fechas/${fechaInicial}/${fechaFinal}`);
```

Esta es una petición que se hace al servlet de reporte, mando una instrucción para manejar ese caso “entre-fechas” y después se procede a mandar un par de filtros, que en este caso son fechas pasadas como string.

En la parte del backen pasa los siguiente:

Primero se procede a recibir la información que viene de la petición

```
String path = request.getPathInfo();
```

Si el path recibido no es nulo o si no solamente trae la diagonal, quiere decir que efectivamente trae información que puede procesarse, acá si la petición tiene escrito entre-fechas, entonces ya se conoce a que if es al que irá dirigido.

```
if (path != null && !path.equals("/")) {
    String[] partes = path.substring(1).split("/");
    if (partes[0].equals("entre-fechas")) {
        String fechaInicial = partes[1];
        String fechaFinal = partes[2];
```

```
        List<ReservacionDB> reservaciones =
reportedao.getReservacionesEntreFechas(fechaInicial, fechaFinal);
        response.getWriter().write(gson.toJson(reservaciones));
```

Acá espera específicamente 2 campos los cuales son fechas porque en esta petición tiene la intención de traer las reservaciones entre un rango X de fechas.

En la parte final se utiliza la lista de ReservacionDB y se escribe en un formato json, importante, los nombres que tiene la clase de Reservacion, deben coincidir con nombres del objeto que se supone recibirá los datos en el frontend, estos objetos son escritos en el objeto HttpServletResponse con ayuda de un objeto Gson que los transforma a un formato json legible para typescript.

1.2. Peticiones de tipo Post

Pasando a los post, estos solo entran a la petición Servlet de tipo Post, no es común mandar algo por la petición pero tiene la diferencias que espera un elemento como body, estos objetos por lo general no esperan algún retorno así que solo esperan void

```
public crearReservacion(nuevo: Reservacion): Observable<void> {
    return this.httpClient.post<void> (`${this.restApi.getApiURL()}reservacion`, nuevo);
```

```
}
```

Este tipo de petición tiene su diferencia en el backend, ya que el objeto de tipo json enviado desde el frontend, es transformado a un objeto de java apoyándose de un objeto Gson, de igual forma los nombres que vienen desde el frontend deben de coincidir exactamente con los nombres del objeto de java, si no estos no serán leídos por el método.

```
PagoRequest pagorequest = gson.fromJson(request.getReader(),  
PagoRequest.class);
```

1.3. Peticiones de tipo Put

Las peticiones de tipo put funcionan como una combinación de los dos anteriores, por lo general se manda un complemento en el path de la petición como el id del objeto a manipular, junto a un objeto que serán los valores que se pretenden cambiar o actualizar dentro de la base de datos.

```
public actualizarReservacion(actual: Reservacion): Observable<void> {  
    return this.httpClient.put<void> (`${this.restApi.getApiURL()}reservacion`, actual);  
}
```

1.4. Peticiones de tipo Delete

Las peticiones de tipo delete son aún más sencillas, ya que solo sería necesario mandar un id dentro del path, y con una consulta de tipo DELETE eliminar el objeto que coincida con el id

```
public eliminarPaquete(idPaquete: number): Observable<void> {  
    return this.httpClient.delete<void> (  
`${this.restApi.getApiURL()}paquete/${idPaquete}`);  
}
```

Si hablamos de la aplicación del Proyecto 1 de IPC2, puede hacer todas estas opciones con casi todos los objetos, salvo puntos contados que no tendrían sentido afectarlos de esa forma.

1.5. Angular y su uso de los FormGroup y FormControl

Una muy buena utilidad de angular, es que utiliza objetos que funcionan de manera reactiva, es decir, que se actualizan en tiempo real cuando el usuario interactúa con ciertos elementos, por ejemplo un input, si el input está seleccionado por un FormControl, cuando en el input se escriba algo, el formcontrol reacciona y guarda inmediatamente el valor en un atributo, así no es necesario actualizarlo en cada ocasión con un botón y el FormGroup es un objeto que almacena objetos de FormControl, esto facilita la obtención de objetos y su guardado porque al obtener los valores con el método .value, se obtienen en formato json, así que en ocasiones ya ni siquiera es necesario transformarlo a un objeto de un modelo de objeto guardado.

1.6. Login

Al inicio del programa, lo que se encuentra es un login, este hace uso de un FormGroup para obtener los valores del usuario posteriormente hace una petición de tipo post, porque de ser un Usuario de Atención al cliente, este debe de registrar el id del usuario porque es un campo necesario para la construcción de una reservación que es utilizado al momento de reportes

1.7. Registro de Usuarios

Los usuarios llenan el formulario con los campos respectivos y al elegir el tipo de usuario, es el punto de partida para las opciones que les proporcionará la página.

1.8. Carga si existe Registro

Existe puntos en donde al solo poder existir un solo registro, primero se intenta cargar esos valores si es que existen, si no entonces se procede a permitir la acción de creación de uno, por ejemplo en la propuesta de un proyecto, solo puede existir un registro, si existe se carga y la petición en lugar de creación es de modificación, al igual que con la calificación de un cliente a un freelancer en un proyecto, en esa entrega el cliente puede cambiar su calificación si así lo desea, pero si el mismo freelancer trabaja en un proyecto diferente del mismo cliente, se habilita una nueva calificación, la calificación está ligada a un proyecto y no de un cliente a un freelancer.

1.9. Inhabilitación/habilitación de Cuentas

El administrador puede con un solo botón descontinuar o habilitar la cuenta, el botón es directo así que al presionarlo una vez, cambia el estado directamente, aunque se ve identificado por un color y palabra representativa.